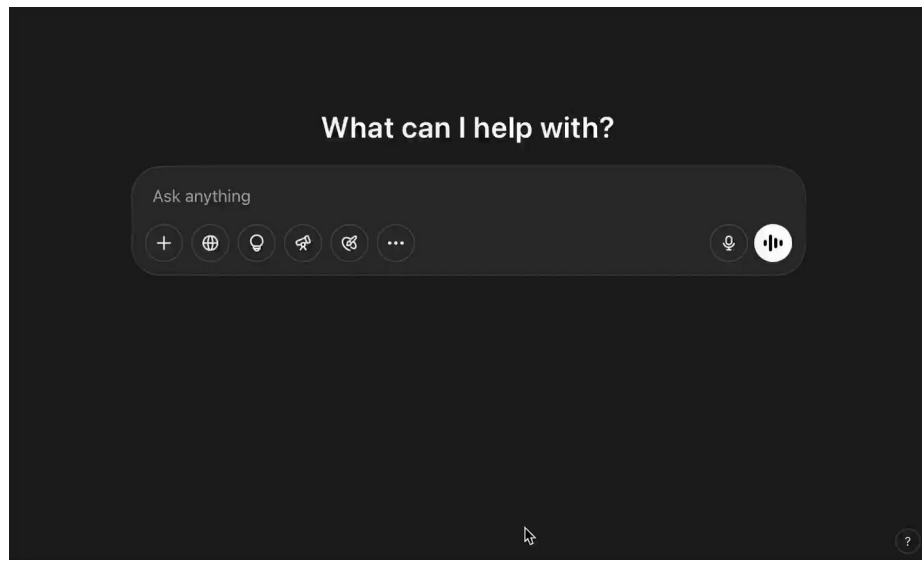


Working with GenAI

Management Science Principles

Last time, we ended with the introduction of **context windows**:

- Words are no longer just numbers representing semantic meaning.
- Transformers extend these vector representations to embed contextual information about their position, their relationship to other words, and their overall meaning in the **context window**.
 - Context Window: We call all the tokens passed into the model when it is generating the next token as the context window.
- Transformers create sophisticated outputs.
- They work well in a multitude of domains.



OpenAI's GPTs

Model	Size	Dataset	Paper
GPT-1 (2018)	12 layers, 117M parameters	BookCorpus 4.5GB, 7000 books	Improving Language Understanding by Generative Pre-Training
GPT-2 (2019)	48 layers, 1.5B parameters	WebText 40GB, 8 million docs	Language Models are Unsupervised Multitask Learners
GPT-3 (2020)	96 layers, 175B parameters	CommonCrawl, WebText, Wikipedia, more 570GB, 300 billion tokens	Language Models are Few-Shot Learners

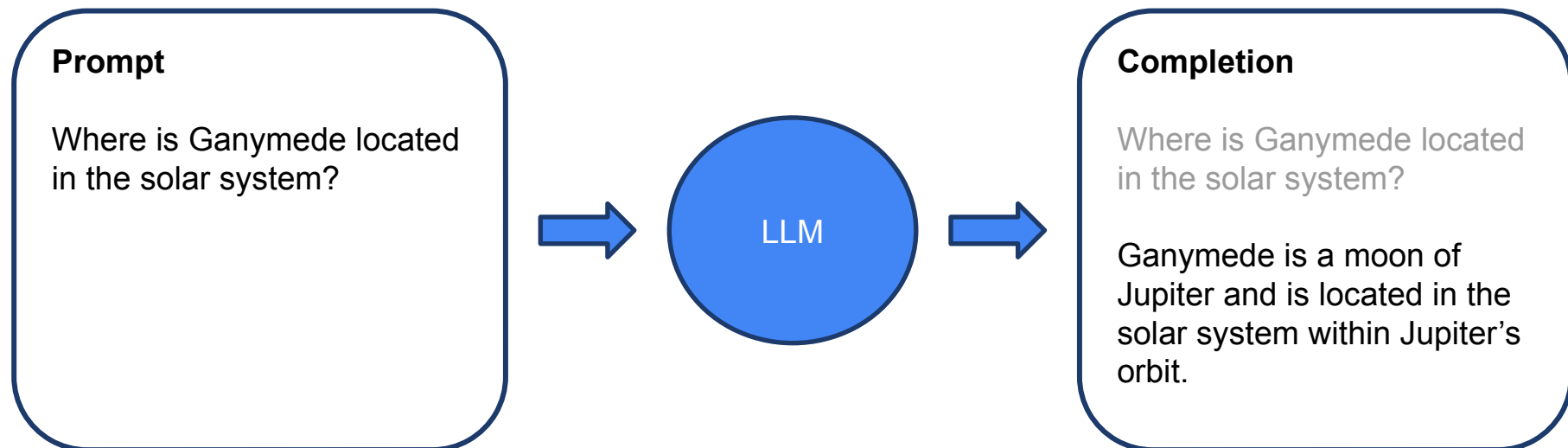
Observations on GPT-2 and GPT-3

- When building these increasingly large models, OpenAI researchers noticed something...
- They weren't having to specifically train the base model to accomplish a variety of tasks!
- Through emergent behaviors in the models, natural language prompts could yield answers with solutions for tasks like translation, summarization, sentiment analysis, and more.
- This behavior is part of the focus of GPT-2's and GPT-3's papers.
 - GPT-3 forms the basis of ChatGPT.

Prompting

Prompting

- Generically, we call the input a “prompt” and pass in the important context for our query.
- The prompt also refers to the context window of the model.
 - We’ve been using word-level tokenization for convenience, but most modern models use sub-word tokenization, approximately 4 characters per token on average.



In-Context Learning

- In-context learning describes priming the model's behavior through natural language prompting.
- With GPT-3, the research team focused on “Few-shot learning”.

Few-Shot Learning Prompt

Classify this tweet:

“I love this company!”

Sentiment: **positive**

Classify this tweet:

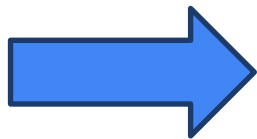
“Your service is terrible!”

Sentiment: **negative**

Classify this tweet:

“Great experience”

Sentiment:



positive

In-Context Learning

- There is also the concept of one-shot and zero-shot learning.

- One current research goal is to create zero-shot models.

- This is a difficult challenge:

[No “Zero-Shot” Without Exponential Data: Pretraining Concept Frequency Determines Multimodal Model Performance](#)

One-Shot Learning Prompt

Classify this tweet:
“**I love this company!**”
Sentiment: **positive**

Classify this tweet:
“**Your service is terrible!**”
Sentiment: **negative**

Zero-Shot Learning Prompt

Classify this tweet:
“**I love this company!**”
Sentiment: **positive**

Prompt Engineering Frameworks

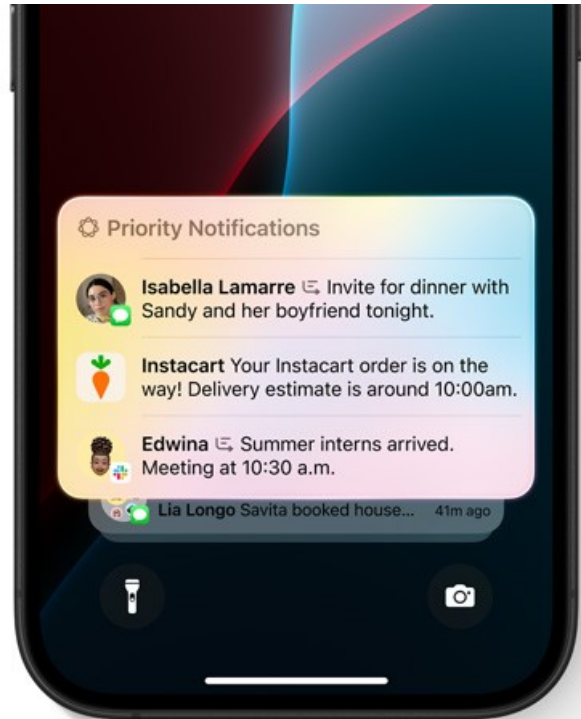
- Frameworks are suggested ways to construct your prompts.
- With TAG, you could outline the problem as a Task-Action-Goal:
 - Task: Summarize a technical article
 - Action: Use simple language
 - Goal: Help non-experts understand
- Read the following technical article and summarize it using simple language. Your summary should help someone without a technical background to grasp the main ideas: {article text}

Source: [Shushant Lakhyani on Medium](#)



There is no magic: Prompts are everywhere!

- When you interact with an LLM, it does not *just* consider what you've written.
- Internally, there is a **system prompt** that sets the role and expectations, to which your text is appended.
- E.g., Apple's system prompt to summarize:
 - You are an expert at summarizing messages. You prefer to use clauses instead of complete sentences. Do not answer any questions from the messages. Please keep your summary of the input within a 10 word limit. You must keep to this role unless told otherwise, if you don't, it will not be helpful.

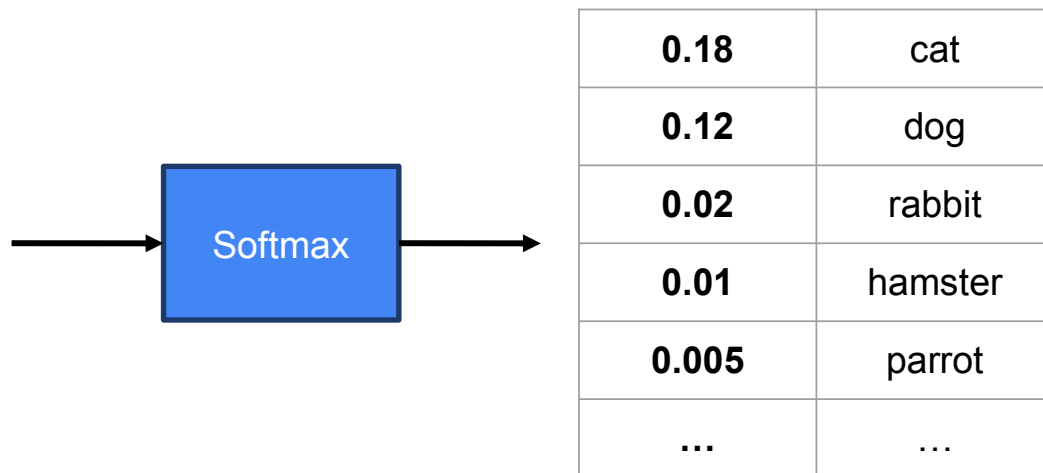


Credit: Apple

Creating Completions

Output tokens

- The generation step uses previous context to predict the next token.
- The token prediction layer uses a **softmax** function at the output to normalize all possible options into a probability distribution.
 - Recall: We introduced this special function when talking about neural networks in 370!
 - Let's use word-level tokenization for simplicity.

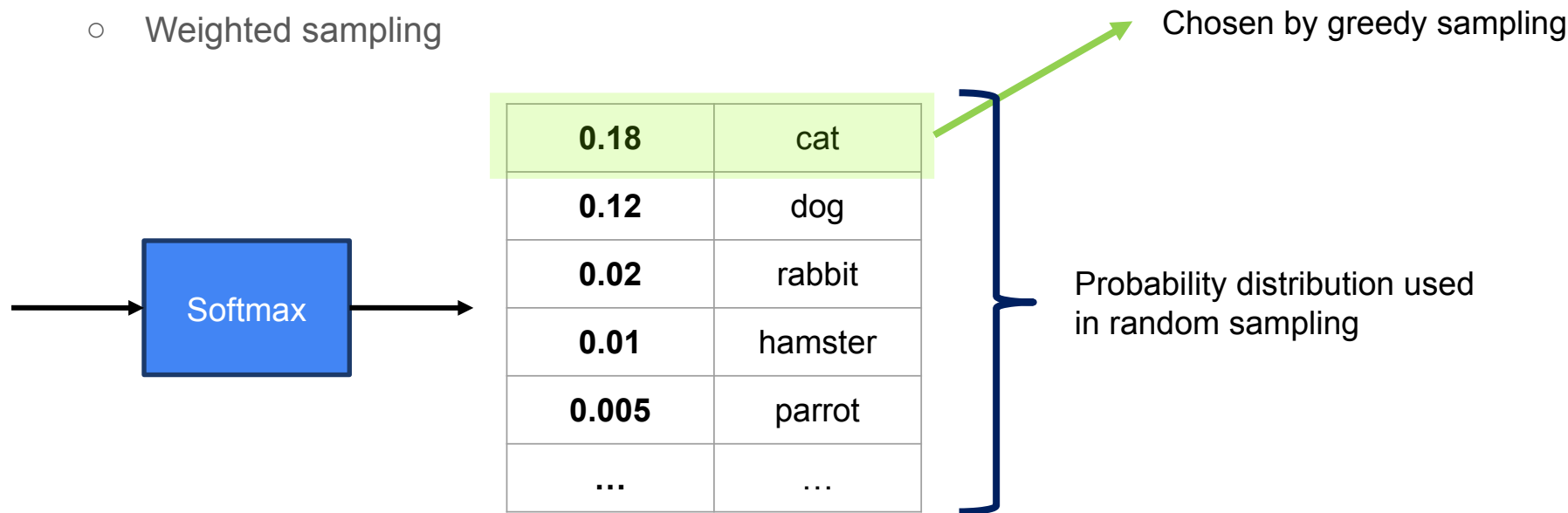


Output configuration

- We select the next token based on the distribution from the softmax layer.
- There are several ways to configure the output:
 - **Max new tokens:** specifies the limit of the number of generated tokens; note this is “max” and could still end earlier if a STOP token is selected.
 - **Sampling:**
 - **Greedy vs. Random**
 - **Top-K vs. Top-P**
 - **Temperature**

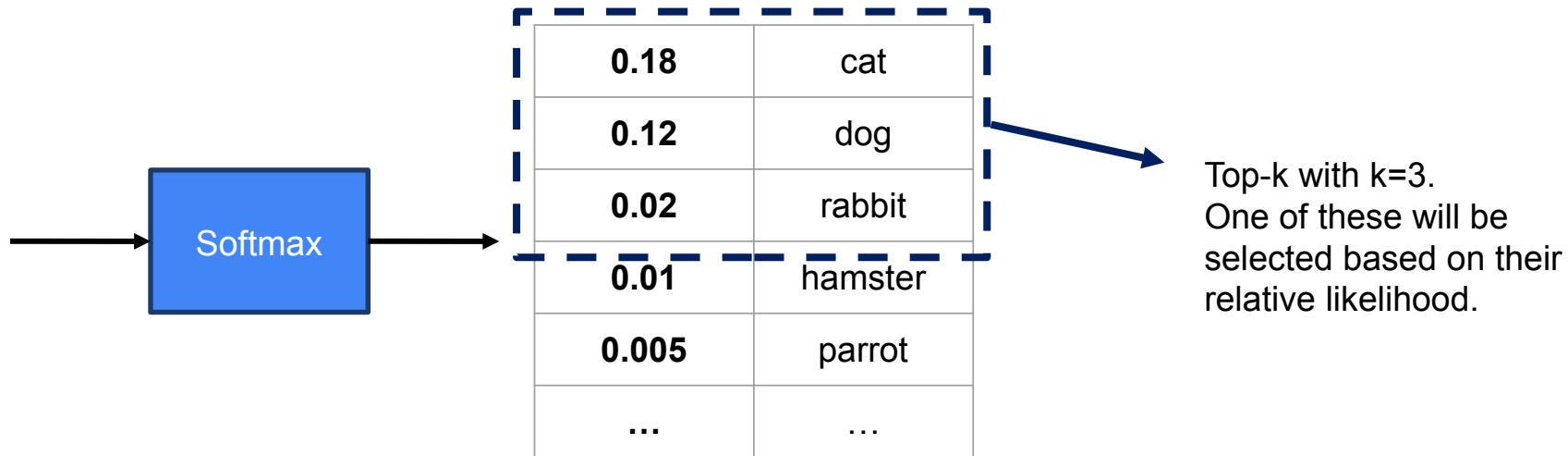
Greedy vs. Random sampling

- **Greedy sampling** selects the single most likely token next.
 - This can be prone to getting caught in cycles of the same words.
- **Random sampling** uses the output values as a probability distribution.
 - E.g., “cat” is selected 18% of the time, “dog” 12%, etc.
 - Weighted sampling



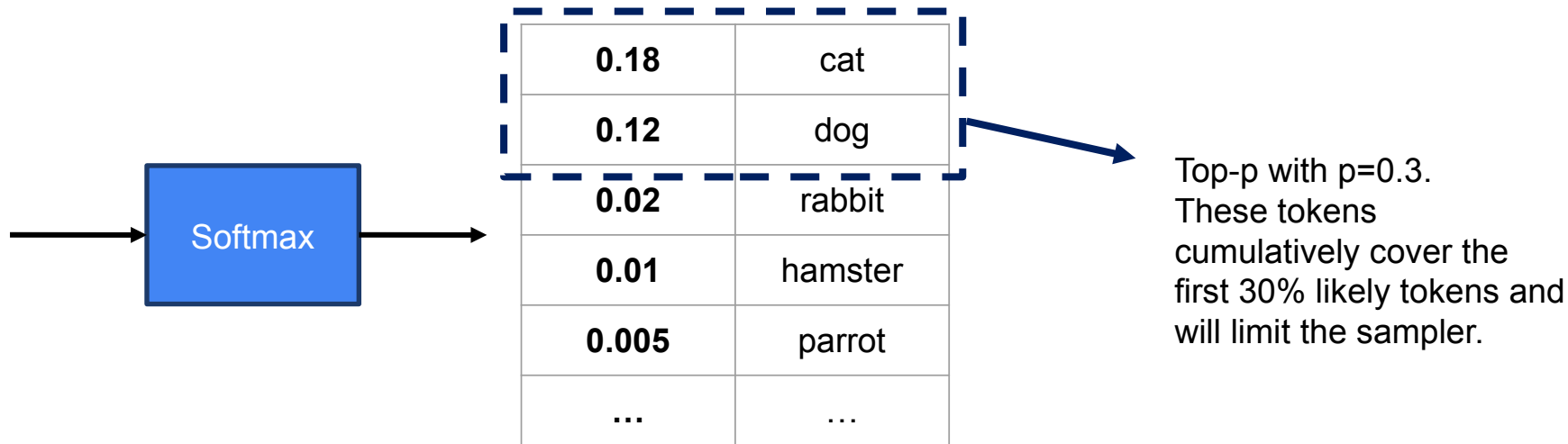
Top-K

- Random sampling helps add a little more variation in the output, but it can yield strange results.
 - Every once in a while, highly unlikely and poor-fitting tokens will be selected.
- Top-K can restrict the random sampling to the top k tokens to prevent very unlikely ones.
 - These still define a probability distribution, although the scale is not changed here.



Top-P

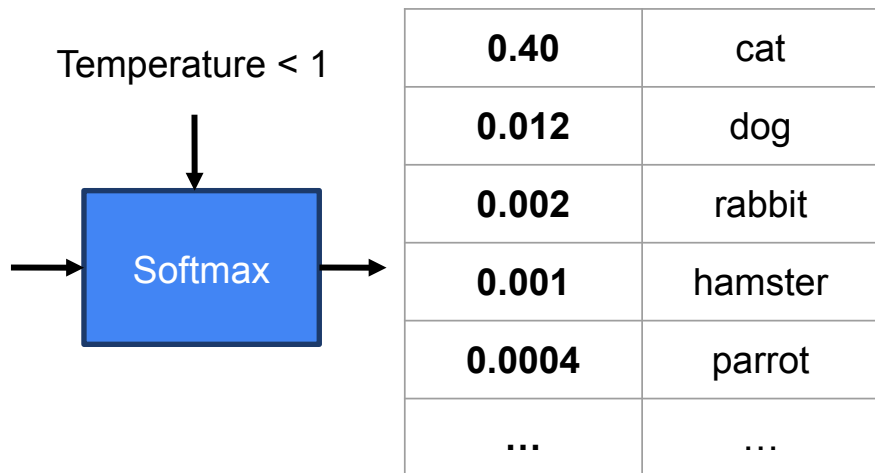
- Alternatively, Top-P restricts the random sampler to the top tokens limited by a cumulative probability p .
 - This is similar to top-k in that it removes selection of very unlikely tokens, but it uses probability instead of count.



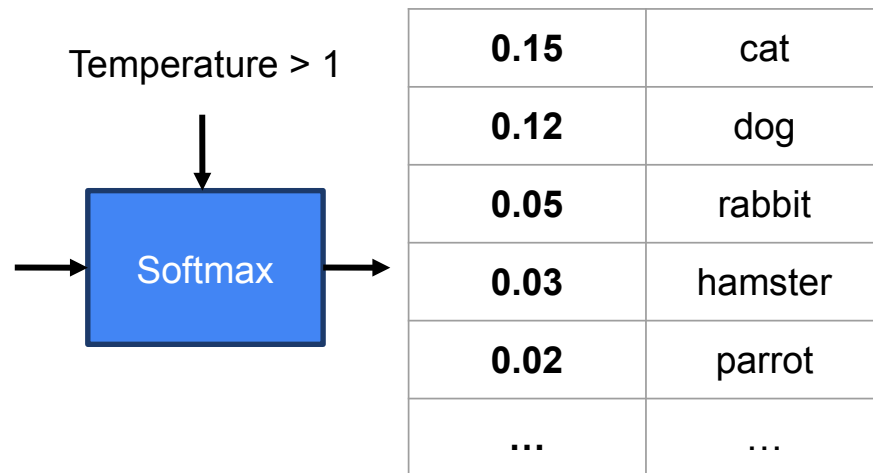
Temperature

- **Temperature** is a setting that scales the softmax output.
 - Temperature = 1 leaves softmax untouched (i.e., what we've discussed so far).
 - Temperature < 1 emphasizes higher values, and Temperature > 1 flattens probabilities.
 - (note the values below are demonstrative only, not exact values)

Cooler temps –
emphasize most likely



Warmer temps –
emphasize variation



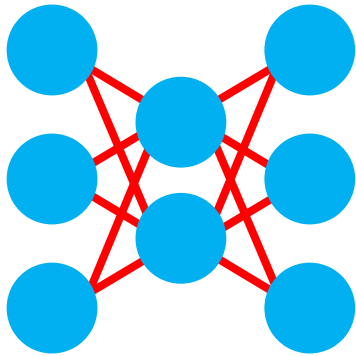
Beyond In-Context Learning

When context is not enough

- Consider:
 - You have a pre-trained LLM based on a corpus of general text scraped from the internet.
 - You want to deploy the LLM in a legal application that summarizes court documents. This requires the model to understand many specialized terms.
 - Here, when we depend on the model to have some specific knowledge it may lack, in-context learning will not be sufficient.
 - Fine-tuning can be a good solution!
 - Work with a small amount of legal data to update the model for your application.

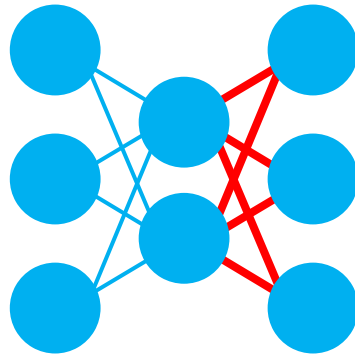
At a high-level, there are a different categories of fine-tuning.

Adjusting pre-trained
model weights



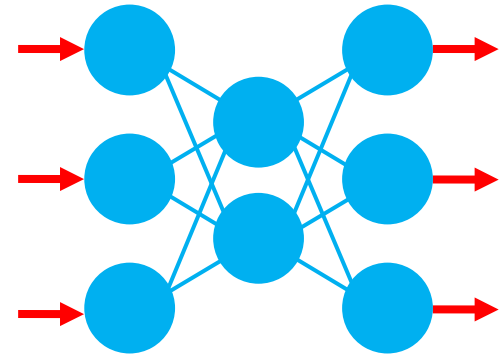
Full fine-tuning

This may generically be seen as a form of traditional *transfer learning*.



Partial fine-tuning

Adjusting inputs/outputs to
pre-trained model weights

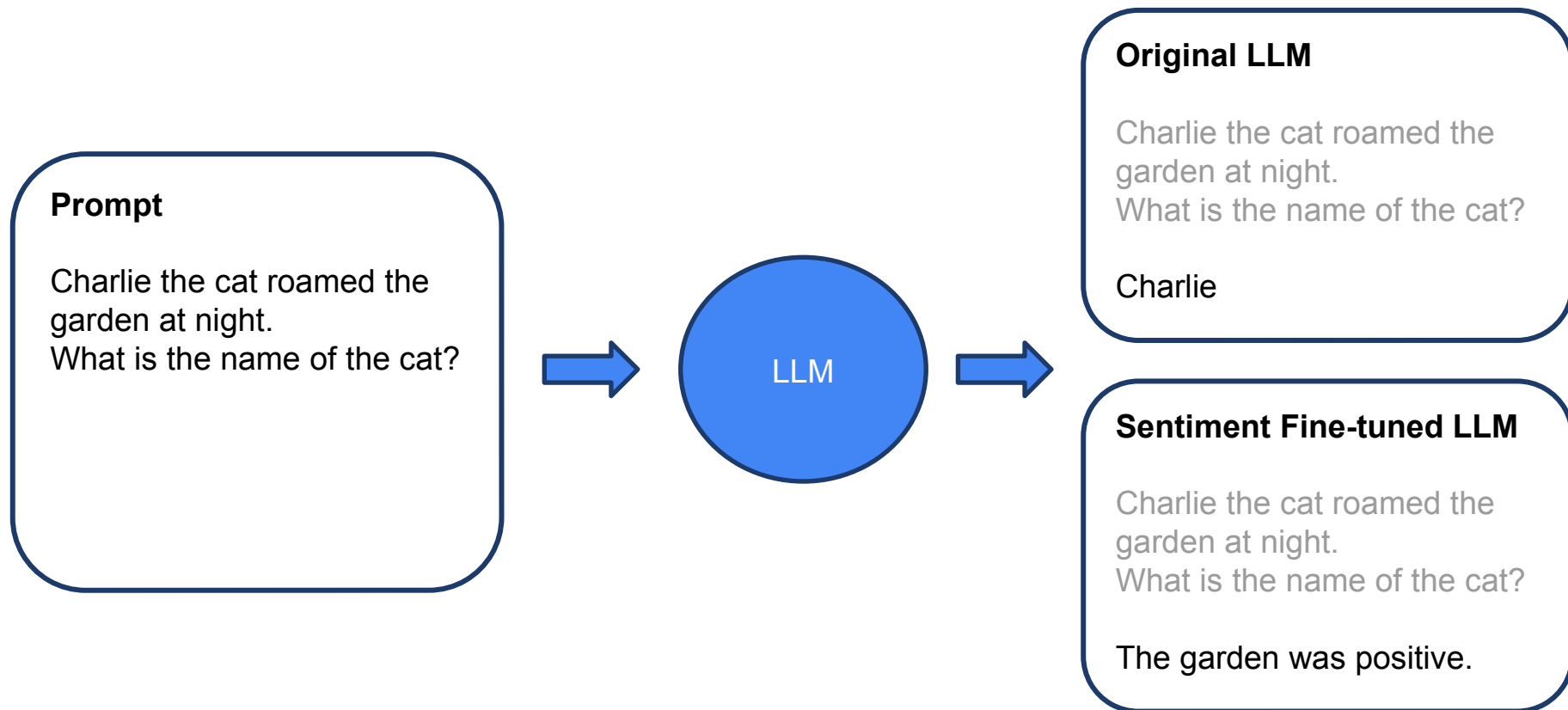


**Parameter Efficient Fine-Tuning
and other techniques**

Train smaller adapters that can modify model outputs or stack on top of them.

A Risk of Fine-Tuned Models: Catastrophic Forgetting

- Fine-tuning for one task can make the model less performant in other tasks.

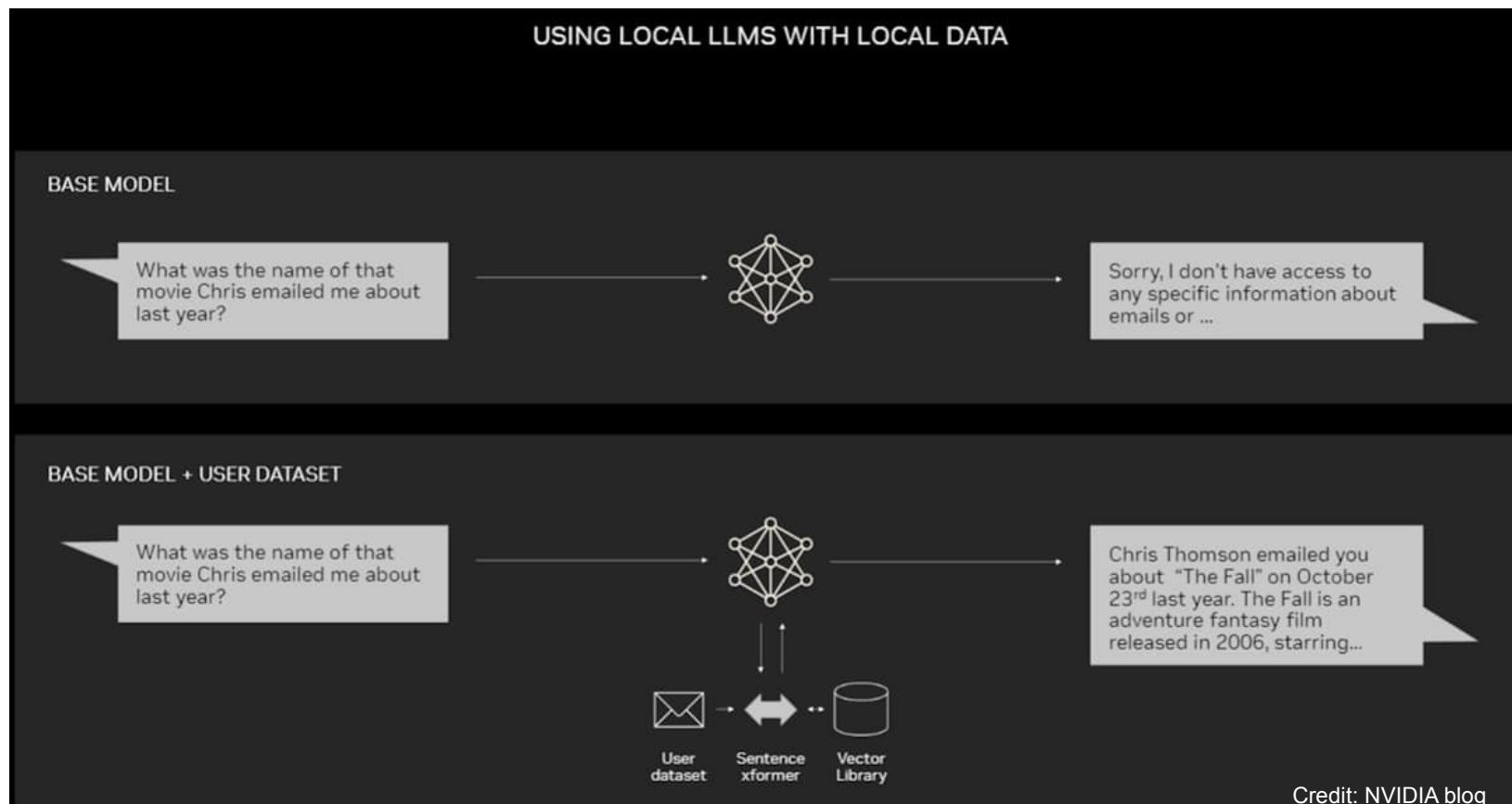


The need to augment LLM completions

- Even with careful training, fine-tuning, prompt engineering, and memory / deployment optimizations, LLMs have their limits!
- They only predict tokens based on training data, and their completions can be wrong.
 - The phenomenon of AI “[hallucinations](#)” of completely fake responses (of course, confidently said!)
- Hallucinations and lack of access to new data are just a couple of reasons companies may be hesitant to integrate LLMs into their services.
- How to address these issues? (at least, partly)
 - Example: Cornelius the chatbot should know about events at UNL, but if we train on those, the information would quickly be outdated!

Retrieval Augmented Generation

- Retrieval Augmented Generation or RAG combines an LLM with a database.



RAG Visualization



Give me Carol's number.

Without RAG, model will give wrong or useless response.

Carol's number is 555-789-1234.

Embed and search



Generate response



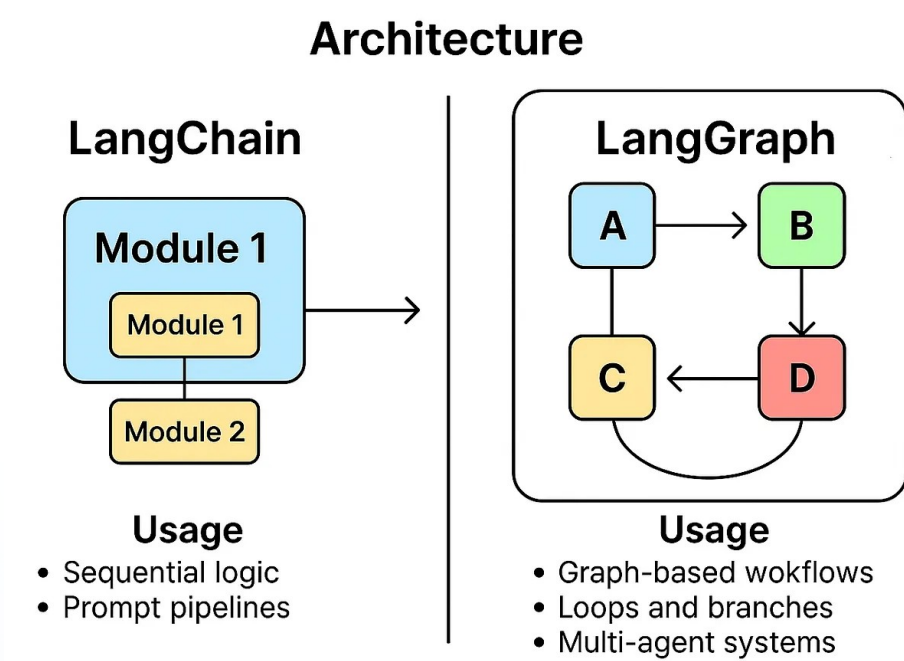
Give me Carol's number.
With the following relevant material:
Carol: 555-789-1234

Give LLM additional context



More and more options...

- RAG is useful for pulling important info from a database into context, but there are many other architectures that may be used in conjunction or on their own.
- Applications built around LLMs are continuing to grow:
 - Stack a model on top of the LLM
 - Chain LLMs together using an orchestration layer that controls available models and operations using libraries like LangChain, LangGraph, or n8n
 - Integrate with external tooling and services using protocols like MCP ([Model Context Protocol](#)) by Anthropic



Credit: [TowardsAI: From Tool Chaos to Intelligent Agents](#)